CURRICULUM VITAE

Kalen Patterson

AI SYSTEMS ENGINEER

kcpatterson27@gmail.com · github.com/kcpatt27 · portfolio

Summary

AI systems engineer who builds local-first tools end to end: agent infrastructure, MCP tooling, decision engines, and grounded LLM pipelines. Strong TypeScript and Python, with a bias toward tests, evidence, and measurable results. Pursuing a B.S. in Computer Science at Western Governors University.

Skills

Languages	TypeScript, Python, JavaScript, SQL, Go
Frontend	React, Next.js, Vite, Tailwind, D3.js, HTML/CSS
Backend and data	Node.js, FastAPI, PostgreSQL, SQLite, Prisma, Supabase, Cloudflare Workers
AI and agents	MCP, Ollama, OpenRouter, local inference (ONNX), agent orchestration, grounding and evaluation
Practices	Docker, GitHub Actions, Playwright, pytest, Vitest, Tailscale, Git

Experience

Software Engineer, NextDish

2024–2026 · Private

Next.js, TypeScript, Supabase, Cloudflare, OpenRouter

- Architected and solo-built a full-stack AI culinary platform: 131+ React components, ~40 API endpoints, a 20+ table schema, and 339 commits over 14 months.
- Implemented hybrid AI generation: a client-side Llama 3.2 model (ONNX) for offline, zero-cost generation, plus an OpenRouter server tier with dynamic model discovery and automatic fallback.
- Hardened the data layer by refactoring 50+ Supabase row-level-security policies across 20 tables and removing 12 unused indexes, clearing all performance-advisor warnings.

Selected projects

eyewright

Python, MCP, local decision models

- Published a decision-engine package (PyPI v0.4.0) exposing a committee of local models behind one callable, reaching 88.9% acted accuracy at 22.5% coverage with all traffic local by default.
- Ran a 2,000-decision model-selection replay across 5 models and 4 committee designs, proving naive majority voting underperforms the best single model; redesigned the committee as a calibrated confidence-and-consult cascade.

AI Security Layer

TypeScript, PostgreSQL, Ed25519

- Built a high-assurance agent-execution environment (~14k LOC, 716 tests) enforcing “no action without capability and policy”: 20/20 adversarial requests denied and 0/4 prompt-injection successes.
- Designed a tamper-evident audit plane (SHA-256 hash chain, per-event Ed25519 signatures, external anchors) with capability delegation, Postgres-enforced tenant isolation, and a deterministic SBOM diff gate.

control-plane

TypeScript, SQLite, agent orchestration

- Designed and built an agent-orchestration system (459 tests) that runs coding agents across ~10 repositories behind one state layer: 296 commits in 5.5 weeks, verification-gated autonomy, per-dispatch and monthly budget ceilings, and a crash-safe supervisor loop.
- Enforced a trust model where agent claims are never truth: external verification (tests, typecheck, lint, build) decides every task outcome, every merge to main is human-gated, and a deterministic model router records auditable reasons per dispatch.

Living State Machine

Python, FastAPI, Ollama, LanceDB

- Built a local-first LLM memory system implementing six ablatable biologically inspired mechanisms, reaching 1.000 vs 0.100 preference-recall accuracy over naive RAG at matched context budget (10× baseline across 3 seeds).
- Added a deterministic verifier and a gated writer (F1 0.98) that proved critical under adversarial decoy load, with a 240+ test suite and a YAML-reproducible multi-seed ablation runner.

Education

Western Governors University

B.S. Computer Science · Pursuing

Additional work

Also built a daily-driver orchestration CLI for a fully local AI stack (907 tests; cut agent latency from ~25–60 s to ~2–5 s), a privacy-first AI code editor, a published CLI that audits MCP context-token cost (458 downloads in its first week), an interactive D3 dashboard covering 240+ countries, and an offline language-learning app with Lighthouse accessibility 1.0 across six pages. Author of a 25,000-word published guide on building software with AI. Built with AI assistance.